RKNN2 SDK Quick Start

ID: RK-JC-YF-416

Release Version: V2.5.0

Release Date: 2026-09-03

Security Level: □Top-Secret □Secret □Internal ■Public

DISCLAIMER

THIS DOCUMENT IS PROVIDED "AS IS". ROCKCHIP ELECTRONICS CO., LTD.("ROCKCHIP")DOES NOT PROVIDE ANY WARRANTY OF ANY KIND, EXPRESSED, IMPLIED OR OTHERWISE, WITH RESPECT TO THE ACCURACY, RELIABILITY, COMPLETENESS, MERCHANTABILITY, FITNESS FOR ANY PARTICULAR PURPOSE OR NON-INFRINGEMENT OF ANY REPRESENTATION, INFORMATION AND CONTENT IN THIS DOCUMENT. THIS DOCUMENT IS FOR REFERENCE ONLY. THIS DOCUMENT MAY BE UPDATED OR CHANGED WITHOUT ANY NOTICE AT ANY TIME DUE TO THE UPGRADES OF THE PRODUCT OR ANY OTHER REASONS.

Trademark Statement

"Rockchip", "瑞芯微", "瑞芯" shall be Rockchip's registered trademarks and owned by Rockchip. All the other trademarks or registered trademarks mentioned in this document shall be owned by their respective owners.

All rights reserved. ©2026. Rockchip Electronics Co., Ltd.

Beyond the scope of fair use, neither any entity nor individual shall extract, copy, or distribute this document in any form in whole or in part without the written approval of Rockchip.

Rockchip Electronics Co., Ltd.

No.18 Building, A District, No.89, software Boulevard Fuzhou, Fujian, PRC

Website: www.rock-chips.com

Customer service Tel: +86-4007-700-590

Customer service Fax: +86-591-83951833

Customer service e-Mail: fae@rock-chips.com

Preface

Overview

This document is intended for users who are using the RKNN2 SDK for the first time. Its goal is to complete environment checks, model conversion, on-board debugging, and board deployment via the shortest path. For the complete software stack description, model development workflow, API reference, and systematic troubleshooting, please refer to the corresponding main documents.

Intended Audience

This document is mainly intended for:

Technical support engineers

Software development engineers

Revision History

Version	Modifier	Date	Modify description	Reviewer
V1.6.0	HPC	2023-11-28	Initial version	Vincent
V2.0.0-beta0	HPC	2024-03-15	Add RK3576 content	Vincent
V2.1.0	HPC	2024-08-01	Use Miniforge to replace MiniConda	Vincent
V2.2.0	HPC	2024-09-04	Update version number	Vincent
V2.3.0	HPC	2024-11-06	Add support for installing RKNN-Toolkit2 via pip source	Vincent
V2.3.2	HPC	2025-04-03	Add support for RV1126B	Vincent
V2.5.0	HPC	2026-09-03	Document restructuring	Vincent

Table of Contents

1. Before You Start
    - 1.1 Guide Objectives
    - 1.2 Software Framework and Key Components
    - 1.3 Select the Example
    - 1.4 Supported Platforms and Version Requirements
    - 1.5 Obtain the SDK and Companion Repositories
2. Hardware Connection and Checks
    - 2.1 Hardware Checklist
    - 2.2 Introduction to the Development Board and Connection Tools
    - 2.3 Connect the Development Board
    - 2.4 Check Device Connection Status
3. Host and Board Environment Preparation
    - 3.1 Install RKNN-Toolkit2
        - 3.1.1 Install the Python Environment
        - 3.1.2 Install RKNN-Toolkit2
    - 3.2 Verify the Toolkit Installation
    - 3.3 Install or Check the RKNPU2 Environment on the Board
        - 3.3.1 Confirm the NPU Driver Version
        - 3.3.2 Check rknn_server and the Runtime Library
        - 3.3.3 Install/Update the RKNPU2 Environment
    - 3.4 Install Compilation Tools
        - 3.4.1 Install CMake
        - 3.4.2 Install Compilers
            - 3.4.2.1 Confirm the System Type and Architecture of the Development Board
            - 3.4.2.2 Install NDK on the Android System Development Board
            - 3.4.2.3 Install the GCC Cross-Compiler on the Linux System Development Board
    - 3.5 Environment Pre-Check
4. Run Example Models
    - 4.1 Run the Image Classification Example
        - 4.1.1 Convert the Model
        - 4.1.2 Run the RKNN Python Demo
        - 4.1.3 Run the RKNN C Demo
    - 4.2 Run the Object Detection Example
        - 4.2.1 Convert the Model

4.2.2 Run the RKNN Python Demo

4.2.3 Run the RKNN C Demo

4.2.4 Dataset Accuracy Evaluation (Optional)

5. Run RKNN Python Demo in Docker (Optional)

5.1 Install Docker

5.2 Prepare the RKNN-Toolkit2 Image

5.3 Run the RKNN Python Demo in Docker

6. Common Startup Issues and Next Steps

6.1 adb devices Shows No Device

6.2 Manually Start the rknn_server Service

6.3 Document Navigation

1. Before You Start

This guide uses the shortest path to walk you through the first run of the RKNN2 SDK, covering hardware connection, environment preparation, model conversion, on-board debugging, and board deployment. Model conversion principles, quantization configuration, the complete API reference, and systematic troubleshooting are beyond the scope of this guide. Please refer to the corresponding documents.

> **Note:** For the complete development workflow, Toolkit interface descriptions, accuracy/performance/memory analysis, etc., please refer to the *RKNN2 SDK Development Guide*. The supported hardware, operating systems, and version ranges are subject to the SDK release package. This guide only lists the necessary information for the first run.

1.1 Guide Objectives

After completing this guide, you should be able to:

1. Confirm that the host computer, development board, and RKNPU2 runtime environment are available;
2. Use RKNN-Toolkit2 to convert an ONNX model;
3. Run the Python Demo on the board via on-board debugging;
4. Compile the C Demo and run it on the board;
5. Navigate to the complete development, API reference, or troubleshooting documents as needed.

1.2 Software Framework and Key Components

The RKNN2 SDK development workflow mainly consists of two parts: the PC-side development kit and the development board runtime environment:

- **PC-side development kit:** You can use RKNN-Toolkit2 to convert models trained with deep learning frameworks such as ONNX, PyTorch, and TensorFlow into RKNN format, and verify the models on the development board via on-board debugging. RKNN Model Zoo provides model conversion and deployment examples.
- **Development board runtime environment:** After model conversion, you can use the RKNPU2 Runtime on the development board to load and run RKNN models.

The key components are as follows:

Component	Running Location	Description
`RKNN-Toolkit2`	PC side	Python toolkit for model conversion, quantization, on-board debugging, and other core features

Component	Running Location	Description
`RKNN Server` (rknn_server)	Board side	Background proxy service that receives commands and data transmitted from the PC, calls the corresponding board-side Runtime interfaces to execute them, and returns the results to the PC
`RKNPU2 Runtime` (librknnrt.so)	Board side	Loads RKNN models and performs inference of RKNN models on the NPU
`RKNN Model Zoo`	PC side / board side	Provides conversion scripts and Python/C++ example code for common models such as YOLOv5, YOLOv8, and MobileNet

1.3 Select the Example

This guide uses the following two examples throughout the first-run workflow:

Example	Purpose	Model Zoo directory
MobileNet	Image classification example	`examples/mobilenet`
YOLOv5	Object detection example	`examples/yolov5`

Each model example provides both Python and C/C++ versions of the sample code. For convenience, they are referred to as the RKNN Python Demo and the RKNN C Demo in the rest of this document. Different Demos may differ in usage. Please follow the steps in the README.md in each directory.

1.4 Supported Platforms and Version Requirements

The supported platforms are as follows:

Category	Description
Hardware platforms	RK3562, RK3566 series, RK3568 series, RK3576 series, RK3588 series, RV1126B
Host computer	Ubuntu 18.04 / Ubuntu 20.04 / Ubuntu 22.04
Python version	3.8 (recommended)
RKNPU2 driver version	>= 0.9.8 recommended

> **Note:** Ensure that the versions of rknn_server, the Runtime library (librknnrt.so), and RKNN-Toolkit2 are consistent. It is recommended to install the latest versions. If you use an RKNN model with dynamic input dimensions, the versions of rknn_server and the Runtime library must be >= 1.5.0.

1.5 Obtain the SDK and Companion Repositories

It is recommended to create a new directory to store the RKNN repositories. For example, create a folder named `Projects` and place the RKNN-Toolkit2 and RKNN Model Zoo repositories in that directory. Refer to the following commands:

```
# Create the 'Projects' folder
mkdir Projects

# Switch to this directory
cd Projects

# Download the RKNN-Toolkit2 repository
git clone https://github.com/airockchip/rknn-toolkit2.git --depth 1

# Download the RKNN Model Zoo repository
git clone https://github.com/airockchip/rknn_model_zoo.git --depth 1

# Notice:
# 1. Parameter --depth 1 means to clone only the latest commit
# 2. If the 'git clone' command fails, you can also download the compressed file
directly from GitHub and unzip it locally in this directory.
```

The default directory structure used in the following examples:

```
Projects
├── rknn-toolkit2
│   ├── doc
│   ├── rknn-toolkit2
│   │   ├── packages
│   │   ├── docker
│   │   └── ...
│   ├── rknpu2
│   │   ├── runtime
│   │   └── ...
│   └── ...
└── rknn_model_zoo
    ├── datasets
    ├── examples
    └── ...
```

2. Hardware Connection and Checks

This chapter only covers the hardware connection and device visibility checks required for the first run.

2.1 Hardware Checklist

The following hardware is required to run the example programs in this document:

Hardware	Quantity	Description
Computer	1	Ubuntu 18.04/20.04/22.04
Development board	1	RK3562/RK3566/RK3568/RK3576/RK3588/RV1126B
Data cable	1	Type depends on the development board, see Section 2.2
Power adapter	1	Specification depends on the development board, see Section 2.2

> **Note:** Due to different production batches, the type and location of the data cable interface of the development board may change. Usually, the interface with the word OTG printed on it is the data cable interface.

2.2 Introduction to the Development Board and Connection Tools

1. Development board

RK3562 development board:

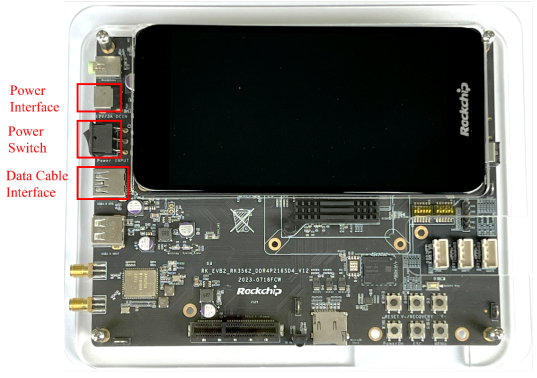

Figure 2-1 RK3562 development board

RK3566 development board:

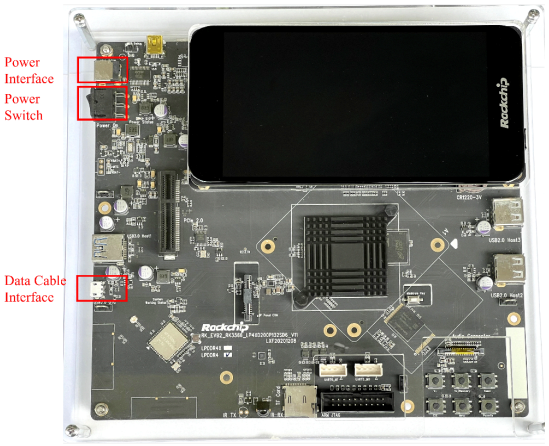

Figure 2-2 RK3566 development board

RK3568 development board:

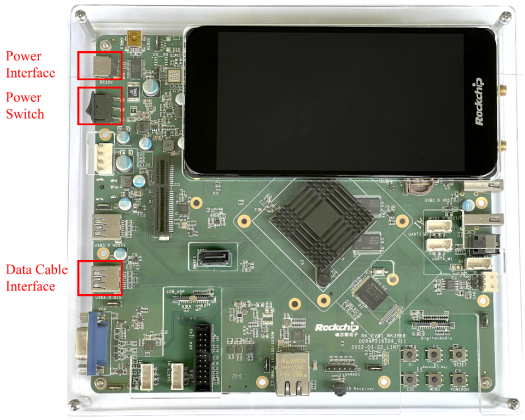

Figure 2-3 RK3568 development board

RK3588 development board:

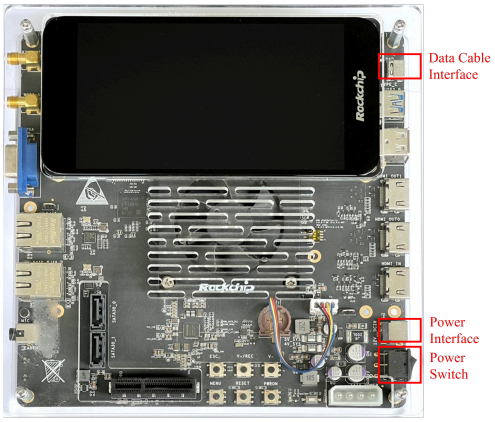

Figure 2-4 RK3588 development board

RK3576 development board:

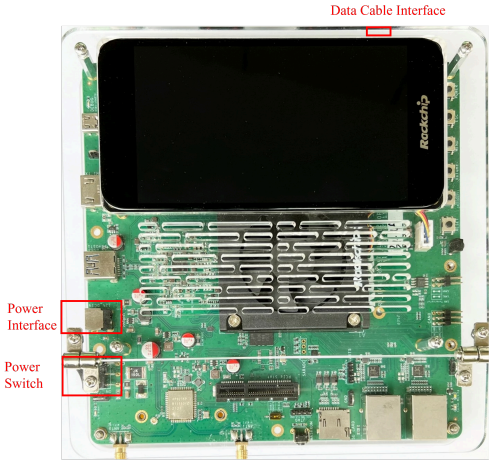

Figure 2-5 RK3576 development board

RV1126B development board:

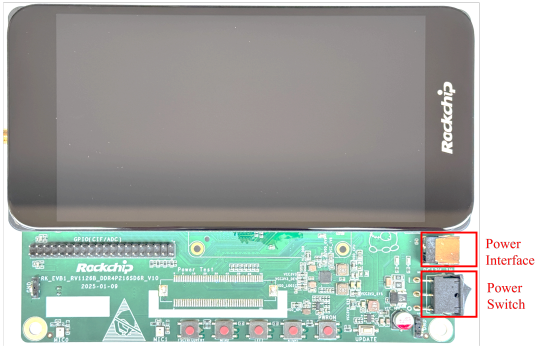

Figure 2-6 RV1126B development board

RV1126B development board (side view):

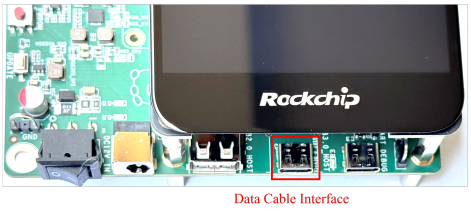

Figure 2-7 RV1126B development board (side view)

2. Data cable

USB-A data cable (for RK3562/RK3568 development boards):

Figure 2-8 USB-A data cable (for RK3562/RK3568 development boards)

Micro USB data cable (for RK3566 development board):

Figure 2-9 Micro USB data cable (for RK3566 development board)

USB-C data cable (for RK3576/RK3588 development boards):

Figure 2-10 USB-C data cable (for RK3576/RK3588 development boards)

3. Power adapter

Power adapter with 12V-2A output (for RK3562/RK3566/RK3568 development boards):

Figure 2-11 Power adapter with 12V-2A output (for RK3562/RK3566/RK3568 development boards)

Power adapter with 12V-3A output (for RK3576/RK3588 development boards):

Figure 2-12 Power adapter with 12V-3A output (for RK3576/RK3588 development boards)

2.3 Connect the Development Board

Taking RK3568 as an example to illustrate how to connect the development board to the computer:

1. Prepare a computer with Ubuntu 18.04 / Ubuntu 20.04 / Ubuntu 22.04 operating system.
2. Find the location of the power interface in the picture and connect the development board power adapter.
3. Use the data cable to connect the development board and computer. (Note that due to different production batches, the type and location of the data cable interface of the development board may change. Usually, the interface with the word OTG printed on it is the data cable interface.)
4. Turn on the power switch and wait for the development board system to start up.
5. Check whether the development board is connected to the computer

In the terminal window (command line interface) of your computer, execute the following command:

```
# If adb has not been installed, please install it first using 'sudo apt install
adb'
adb devices
```

The output information of a successful connection is as follows, where `13af7b28115662cd` is the device ID of RK3568 (example value; the actual value depends on your machine). If there is no device displayed, please refer to 6.1 adb devices Shows No Device for troubleshooting.

```
List of devices attached
13af7b28115662cd    device
```

2.4 Check Device Connection Status

Before continuing with installation and model conversion, make sure that at least all of the following checks pass:

Check item	Command	Pass criteria
ADB tool available	`adb version`	The adb version information is displayed normally
Board device visible	`adb devices`	The development board appears in the device list with the `device` status

After all the above checks pass, continue with the host-side and board-side environment preparation.

3. Host and Board Environment Preparation

3.1 Install RKNN-Toolkit2

This quick start only covers the recommended Conda + pip installation path. For Docker-based installation, please refer to Chapter 5.

3.1.1 Install the Python Environment

If **Python 3.8 (recommended version)** is not installed on your system, or if there are multiple versions of Python installed, it is recommended to use Miniforge Conda to create a new Python 3.8 environment.

Step 1: Install Miniforge Conda

In the terminal window on the computer, execute the following command to check whether Miniforge Conda is installed. If already installed, you can skip this step.

```
conda -V
# Reference output: conda 23.3.1, indicating that the Miniforge Conda version is
23.3.1
# If it shows 'conda: command not found', it means Miniforge Conda is not installed.
```

If Miniforge Conda is not installed, you can download the Miniforge Conda installer from the following link: https://github.com/conda-forge/miniforge/releases/download/25.3.0-1/Miniforge3-25.3.0-1-Linux-x86_64.sh

Then, install Miniforge Conda using the following command:

```
# Copy to the user directory
cp /path/to/Miniforge3-25.3.0-1-Linux-x86_64.sh ~

# Grant executable permissions
chmod 777 Miniforge3-25.3.0-1-Linux-x86_64.sh

# Run the installation script
bash Miniforge3-25.3.0-1-Linux-x86_64.sh
```

Step 2: Create the Python Environment

After installing Miniforge Conda, execute the following command in the terminal window on the computer to switch to the Conda base environment:

```
source ~/miniforge3/bin/activate # The installation directory of Miniforge
# Upon success, the command prompt will change to the following form:
# (base) xxx@xxx:~$
```

Create a Python 3.8 environment named `toolkit2` using the following command:

```
conda create -n toolkit2 python=3.8
```

Activate the `toolkit2` environment. RKNN-Toolkit2 will be installed in this environment:

```
conda activate toolkit2
# Upon success, the command prompt will change to the following form:
# (toolkit2) xxx@xxx:~$
```

> **Note:** The subsequent Python commands are executed in the `toolkit2` environment by default.

3.1.2 Install RKNN-Toolkit2

After activating the `toolkit2` environment, you can install RKNN-Toolkit2 using either the pip source or a local wheel package.

Method 1: Install via pip source

```
pip install rknn-toolkit2 -i https://pypi.org/simple

# If RKNN-Toolkit2 is already installed, you can upgrade it with the following
command
pip install rknn-toolkit2 -i https://pypi.org/simple --upgrade
```

Method 2: Install via local wheel package

```
# Switch to the rknn-toolkit2 directory
cd Projects/rknn-toolkit2/rknn-toolkit2

# Please choose the appropriate requirements file based on the Python version and
processor architecture:
# where cpxx is the Python version
pip install -r packages/x86_64/requirements_cpxx.txt
# pip install -r packages/arm64/arm64_requirements_cpxx.txt

# Install RKNN-Toolkit2
# Please choose the appropriate wheel package file based on the Python version and
processor architecture:
# where x.x.x is the RKNN-Toolkit2 version number, cpxx is the Python version
pip install packages/x86_64/rknn_toolkit2-x.x.x-cpxx-cpxx-
manylinux_2_17_x86_64.manylinux2014_x86_64.whl
# pip install packages/arm64/rknn_toolkit2-x.x.x-cpxx-cpxx-
manylinux_2_17_aarch64.manylinux2014_aarch64.whl
```

3.2 Verify the Toolkit Installation

Keep the `toolkit2` environment activated and execute:

```
python -c "from rknn.api import RKNN; print('RKNN-Toolkit2 import OK')"
```

If the output is:

```
RKNN-Toolkit2 import OK
```

the basic Toolkit import check passes.

If the installation fails, please refer to Section 5.4.2 "RKNN-Toolkit2 Installation Issue" in the *RKNN2 SDK Development Guide* document, which provides detailed solutions for resolving RKNN-Toolkit2 environment installation failures.

3.3 Install or Check the RKNPU2 Environment on the Board

Before running the Demo on the board, make sure that the NPU driver, the rknn_server service, and the Runtime library are correctly installed.

3.3.1 Confirm the NPU Driver Version

You can execute the following command on the board to query the RKNPU2 driver version:

```
dmesg | grep -i rknpu
```

As shown in the following figure, the current RKNPU2 driver version is 0.8.8.

```
rk3588_s_evb7:/ $ dmesg | grep -i rknpu
[    2.919448] [    T1] RKNPU fdab0000.npu: Adding to iommu group 0
[    2.919608] [    T1] RKNPU fdab0000.npu: RKNPU: rknpu iommu is enabled, using iommu mode
[    2.920935] [    T1] RKNPU fdab0000.npu: can't request region for resource [mem 0xfdab0000-0xfdabffff]
[    2.920959] [    T1] RKNPU fdab0000.npu: can't request region for resource [mem 0xfdac0000-0xfdacffff]
[    2.920973] [    T1] RKNPU fdab0000.npu: can't request region for resource [mem 0xfdad0000-0xfdadffff]
[    2.921339] [    T1] [drm] Initialized rknpu 0.8.8 20230428 for fdab0000.npu on minor 1
[    2.924802] [    T1] RKNPU fdab0000.npu: RKNPU: bin=0
[    2.924994] [    T1] RKNPU fdab0000.npu: leakage=10
[    2.925052] [    T1] debugfs: Directory 'fdab0000.npu-rknpu' with parent 'vdd_npu_s0' already present!
[    2.931816] [    T1] RKNPU fdab0000.npu: pvtm=866
[    2.936313] [    T1] RKNPU fdab0000.npu: pvtm-volt-sel=3
[    2.937408] [    T1] RKNPU fdab0000.npu: avs=0
[    2.937608] [    T1] RKNPU fdab0000.npu: l=10000 h=85000 hyst=5000 l_limit=0 h_limit=800000000 h_table=0
[    2.948777] [    T1] RKNPU fdab0000.npu: failed to find power_model node
[    2.948814] [    T1] RKNPU fdab0000.npu: RKNPU: failed to initialize power model
[    2.948826] [    T1] RKNPU fdab0000.npu: RKNPU: failed to get dynamic-coefficient
```

Figure 3-1 RKNPU2 driver version information

The official firmware of Rockchip development boards all installs the RKNPU2 driver. If the above command cannot query the NPU driver version, you may be using third-party firmware, which may not have the NPU driver installed. If you have the firmware source code, you can change the value of the `CONFIG_ROCKCHIP_RKNPU` option in the `kernel config` to 'y' to integrate the NPU driver, then recompile the kernel driver and flash it. It is recommended that RKNPU2 driver version >= 0.9.8.

3.3.2 Check rknn_server and the Runtime Library

The on-board debugging feature of RKNN-Toolkit2 requires the board to have the RKNPU2 environment installed and the rknn_server service running. If the board does not have rknn_server and the Runtime library installed, or if their versions are inconsistent, you need to reinstall the RKNPU2 environment.

In most cases, the development board is already equipped with a version-consistent RKNPU2 environment by default. You can confirm this with the following commands. (If the RKNPU2 environment is not installed or the versions are inconsistent, please follow the steps in Section 3.3.3 to install/update the RKNPU2 environment.)

1. Board running on the Android system

Check if the RKNPU2 environment is installed: if you can start the rknn_server service, it means that the RKNPU2 environment is already installed on the board.

```
# Switch to the board
adb shell

# Start rknn_server
su
setenforce 0
/vendor/bin/rknn_server &
```

If the following output is displayed, it means that the rknn_server service has been successfully started, indicating that the RKNPU2 environment is installed.

```
start rknn server, version: x.x.x
```

Check if the versions are consistent:

```
# Query rknn_server version
strings /vendor/bin/rknn_server | grep -i "rknn_server version"

# Query librknnrt.so library version
# For 64-bit systems
strings /vendor/lib64/librknnrt.so | grep -i "librknnrt version"
# For 32-bit systems
strings /vendor/lib/librknnrt.so | grep -i "librknnrt version"
```

If the following output is displayed, it means that the rknn_server version is x.x.x, and the version of librknnrt.so is x.x.x.

```
rknn_server version: x.x.x
librknnrt version: x.x.x
```

2. Board running on the Linux system

Check if the RKNPU2 environment is installed: if you can start the rknn_server service, it means that the RKNPU2 environment is already installed on the board.

```
# Switch to the board
adb shell

# Start rknn_server
restart_rknn.sh
```

If the following output is displayed, it means that the rknn_server service has been successfully started, indicating that the RKNPU2 environment is installed.

```
start rknn server, version: x.x.x
```

Check if the versions are consistent:

```
# Query rknn_server version
strings /usr/bin/rknn_server | grep -i "rknn_server version"

# Query librknnrt.so library version
strings /usr/lib/librknnrt.so | grep -i "librknnrt version"
```

If the following output is displayed, it means that the rknn_server version is x.x.x, and the version of librknnrt.so is x.x.x.

```
rknn_server version: x.x.x
librknnrt version: x.x.x
```

3.3.3 Install/Update the RKNPU2 Environment

Different board systems require the installation of different RKNPU2 environments. The following sections introduce the installation methods for each system.

> **Note:** If you have already installed a version-consistent RKNPU2 environment, you can skip this section.

1. Board running on the Android system

Switch to the rknpu2 directory, use the adb tool to push the corresponding rknn_server and librknnrt.so to the board, then start rknn_server. Refer to the following commands:

```
# Switch to the rknpu2 directory
cd Projects/rknn-toolkit2/rknpu2

# Switch to root user permissions
adb root

# Remount the file system as read-write
adb remount

# Push the rknn_server executable
# Note: In a 64-bit Android system, BOARD_ARCH corresponds to the arm64 directory;
in 32-bit systems, it corresponds to the arm directory.
adb push runtime/Android/rknn_server/${BOARD_ARCH}/rknn_server /vendor/bin/

# Push the librknnrt.so library
# For 64-bit systems
adb push runtime/Android/librknn_api/arm64-v8a/librknnrt.so /vendor/lib64/
# For 32-bit systems
adb push runtime/Android/librknn_api/armeabi-v7a/librknnrt.so /vendor/lib/

# Switch to the board
adb shell

# Grant executable permissions
chmod +x /vendor/bin/rknn_server

# Restart the rknn_server service
su
setenforce 0
/vendor/bin/rknn_server &
```

2. Board running on the Linux system

Switch to the rknpu2 directory, use the adb tool to push the corresponding rknn_server and librknnrt.so to the board, then start rknn_server. Refer to the following commands:

```
# Switch to the rknpu2 directory
cd Projects/rknn-toolkit2/rknpu2

# Push rknn_server to the board
# Note: In 64-bit Linux systems, BOARD_ARCH corresponds to the aarch64 directory; in
32-bit systems, it corresponds to the armhf directory.
adb push runtime/Linux/rknn_server/${BOARD_ARCH}/usr/bin/* /usr/bin

# Push librknnrt.so
adb push runtime/Linux/librknn_api/${BOARD_ARCH}/librknnrt.so /usr/lib

# Switch to the board
adb shell

# Grant executable permissions
chmod +x /usr/bin/rknn_server
chmod +x /usr/bin/start_rknn.sh
chmod +x /usr/bin/restart_rknn.sh

# Restart the rknn_server service
restart_rknn.sh
```

3.4 Install Compilation Tools

3.4.1 Install CMake

In the computer terminal, execute the following command:

```
# Update package list
sudo apt update

# Install cmake
sudo apt install cmake
```

3.4.2 Install Compilers

3.4.2.1 Confirm the System Type and Architecture of the Development Board

For convenience, the remainder of this document uses "board" to refer to the development board side.

Step 1: Confirm the system type of the board

In the computer terminal, execute the following command:

```
adb shell getprop ro.build.version.release
```

If the output is a number (Android system version), it indicates that the board is running on the Android system.

```
12
```

Otherwise, the board is running on the Linux system.

```
/bin/sh: getprop: not_found
```

Step 2: Confirm the system architecture of the board

If the board is running on the Android system, you can execute the following command on the computer to query the system architecture:

```
adb shell getprop ro.product.cpu.abi
```

The reference output for this command is as follows, where arm64-v8a indicates ARM 64-bit architecture, ABI version 8.

```
arm64-v8a
```

If the board is running on the Linux system, you can execute the following command on the computer to query the system architecture:

```
adb shell uname -a
```

The reference output for this command is as follows, where aarch64 indicates ARM 64-bit architecture.

```
Linux Rockchip 5.10.160 #183 SMP Tue Oct 24 18:52:11 CST 2023 aarch64 GNU/Linux
```

3.4.2.2 Install NDK on the Android System Development Board

> **Scope:** This section is applicable to development boards with the Android system. If the board is running on the Linux system, please skip this section.

- **NDK download URL** (it is recommended to download the r19c version): https://dl.google.com/android/repository/android-ndk-r19c-linux-x86_64.zip
- **Extract the software package**

It is recommended to extract the NDK software package to the `Projects` folder. The location is as follows:

```
Projects
├── rknn-toolkit2
├── rknn_model_zoo
└── android-ndk-r19c # This path will be used when compiling the RKNN C Demo
```

At this point, the path to the NDK compiler is `Projects/android-ndk-r19c`.

3.4.2.3 Install the GCC Cross-Compiler on the Linux System Development Board

> **Scope:** This section is applicable to development boards with the Linux system. If the board is running on the Android system, please skip this section.

- **GCC download URL**
- Board is a 64-bit system: https://developer.arm.com/-/cdn-downloads/permalink/legacy-linaro-gnu-toolchains/6.3-2017.05/gcc-linaro-6.3.1-2017.05-x86_64_aarch64-linux-gnu.tar.xz
- Board is a 32-bit system: https://developer.arm.com/-/cdn-downloads/permalink/legacy-linaro-gnu-toolchains/6.3-2017.05/gcc-linaro-6.3.1-2017.05-x86_64_arm-linux-gnueabihf.tar.xz
- **Extract the software package**

It is recommended to extract the GCC software package to the `Projects` folder. Taking the GCC package for a 64-bit system as an example, the location is as follows:

```
Projects
├── rknn-toolkit2
├── rknn_model_zoo
└── gcc-linaro-6.3.1-2017.05-x86_64_aarch64-linux-gnu # This path will be used when
compiling the RKNN C Demo
```

At this point, the path to the GCC compiler is `Projects/gcc-linaro-6.3.1-2017.05-x86_64_aarch64-linux-gnu/bin/aarch64-linux-gnu`.

3.5 Environment Pre-Check

Before starting model conversion, it is recommended to complete the following checks at once:

```
# Toolkit
conda activate toolkit2
python -c "from rknn.api import RKNN; print('Toolkit OK')"

# Compilation tools
cmake --version

# Devices
adb devices
```

If you need to run the RKNN C Demo, also confirm the compiler path according to the board system:

- Board running on the Android system: confirm the NDK path (e.g., `Projects/android-ndk-r19c`);
- Board running on the Linux system: confirm the GCC compiler path (e.g., `Projects/gcc-linaro-6.3.1-2017.05-x86_64_aarch64-linux-gnu/bin/aarch64-linux-gnu`).

4. Run Example Models

This chapter uses two examples, MobileNet (image classification) and YOLOv5 (object detection), to demonstrate a minimal closed loop of "get the model -> convert -> run on board -> compile and deploy -> run on board -> check the results". For the complete model conversion configuration, quantization methods, accuracy/performance/memory evaluation, and API calling sequence, please refer to the *RKNN2 SDK Development Guide* document.

4.1 Run the Image Classification Example

This section takes the MobileNet model as an example to introduce the conversion, on-board running, and board deployment of the image classification model. The example is located in the `examples/mobilenet` directory.

4.1.1 Convert the Model

Switch to the `rknn_model_zoo/examples/mobilenet/model` directory and run the download_model.sh script to download the MobileNet ONNX model:

```
# Switch to the rknn_model_zoo/examples/mobilenet/model directory
cd Projects/rknn_model_zoo/examples/mobilenet/model

# Download the mobilenetv2-12.onnx model
./download_model.sh
```

Switch to the `rknn_model_zoo/examples/mobilenet/python` directory and run the mobilenet.py script. This script converts the ONNX model to the RKNN model:

```
# Switch to the rknn_model_zoo/examples/mobilenet/python directory
cd Projects/rknn_model_zoo/examples/mobilenet/python

# Usage: python mobilenet.py --model <onnx_model> --target <TARGET_PLATFORM> [--
dtype <i8/fp>]
# Note: Here, the example is for the rk3588 platform. If using a different board,
modify the platform type in the command accordingly
python mobilenet.py --model ../model/mobilenetv2-12.onnx --target rk3588
```

Where `--dtype` defaults to `i8` (8-bit quantization) if omitted, and `fp` means no quantization. After the conversion, this command runs inference directly on the PC simulator without connecting the development board. The converted model is saved as `examples/mobilenet/model/mobilenet_v2.rknn` by default.

4.1.2 Run the RKNN Python Demo

- **Simulator inference:** Without the `--npu_device_test` parameter, inference runs on the PC simulator. The command is the same as in Section 4.1.1, which is suitable for quick verification without a board.

- **On-board running:** Append the `--npu_device_test` parameter to run inference on the board via on-board debugging:

```
python mobilenet.py --model ../model/mobilenetv2-12.onnx --target rk3588 --
npu_device_test
```

After the successful execution of the script, the TOP5 classification results will be output as follows:

```
-----TOP 5-----
[494] score=0.990035 class=n03017168 chime, bell, gong
[469] score=0.003907 class=n02939185 caldron, cauldron
[653] score=0.001089 class=n03764736 milk can
[747] score=0.000945 class=n04023962 punching bag, punch bag, punching ball,
punchball
[442] score=0.000712 class=n02825657 bell cote, bell cot
```

The value in square brackets is the class index, score is the confidence score, and class is the class name.

4.1.3 Run the RKNN C Demo

To run an RKNN C Demo, you need to first compile the C/C++ source code into an executable file, then push it to the board, and finally run it on the board. Before compiling, install the compilation tools first (see Section 3.4): configure ANDROID_NDK_PATH at the beginning of the build-android.sh script for Android boards, or GCC_COMPILER at the beginning of the build-linux.sh script for Linux boards. For the parameters of the build scripts, refer to the README.md in each example directory.

1. Compilation

```
# Board running on the Android system
# Note: configure ANDROID_NDK_PATH at the beginning of the build-android.sh script
cd Projects/rknn_model_zoo
./build-android.sh -t rk3588 -a arm64-v8a -d mobilenet

# Board running on the Linux system
# Note: configure GCC_COMPILER at the beginning of the build-linux.sh script
cd Projects/rknn_model_zoo
./build-linux.sh -t rk3588 -a aarch64 -d mobilenet
```

2. Push Files to the Board

After compilation, the executable file is located in the `rknn_mobilenet_demo` folder under the `install` directory:

```
# Board running on the Android system
adb root
adb push install/rk3588_android_arm64-v8a/rknn_mobilenet_demo /data/

# Board running on the Linux system
adb push install/rk3588_linux_aarch64/rknn_mobilenet_demo /userdata/
```

3. Run the Demo on the Board

```
# Switch to the board
adb shell

# Switch to the rknn_mobilenet_demo directory
# /data/rknn_mobilenet_demo for Android, /userdata/rknn_mobilenet_demo for Linux
cd /userdata/rknn_mobilenet_demo

# Set the library dependency environment
export LD_LIBRARY_PATH=./lib

# Run the executable file
# Usage: ./rknn_mobilenet_demo <model_path> <input_path>
./rknn_mobilenet_demo model/mobilenet_v2.rknn model/bell.jpg
```

The output is in the same TOP5 format as in Section 4.1.2.

4.2 Run the Object Detection Example

This section takes the YOLOv5 model as an example to introduce the conversion, on-board running, board deployment, and accuracy evaluation of the object detection model. The example is located in the `examples/yolov5` directory.

4.2.1 Convert the Model

Switch to the `rknn_model_zoo/examples/yolov5/model` directory and run the download_model.sh script. This script will download an available YOLOv5 ONNX model and store it in the current model directory. Refer to the following commands:

```
# Switch to the rknn_model_zoo/examples/yolov5/model directory
cd Projects/rknn_model_zoo/examples/yolov5/model

# Run the download_model.sh script to download the yolov5 ONNX model
# For example, the downloaded ONNX model is stored at model/yolov5s_relu.onnx
./download_model.sh
```

Switch to the `rknn_model_zoo/examples/yolov5/python` directory and run the convert.py script. This script converts the original ONNX model to the RKNN model. Refer to the following commands:

```
# Switch to the rknn_model_zoo/examples/yolov5/python directory
cd Projects/rknn_model_zoo/examples/yolov5/python

# Run the convert.py script to convert the original ONNX model to RKNN format
# Usage: python convert.py model_path [rk3566|rk3588|rk3562] [i8/fp] [output_path]
python convert.py ../model/yolov5s_relu.onnx rk3588 i8 ../model/yolov5s_relu.rknn
```

Where `i8` enables 8-bit quantization. After the conversion is completed, the corresponding `.rknn` model file is generated in the model directory.

4.2.2 Run the RKNN Python Demo

The RKNN Python Demo runs the YOLOv5 model on the development board via on-board debugging. Switch to the `rknn_model_zoo/examples/yolov5/python` directory and execute the yolov5.py script:

```
# Run the yolov5.py script to run the yolov5 model on the board
# Usage: python yolov5.py --model_path {rknn_model} --target {target_platform} --
img_show

# If the --img_show parameter is included, the result image will be displayed
# Note: Here, the example is for the rk3588 platform. If using a different board,
modify the platform type in the command accordingly
python yolov5.py --model_path ../model/yolov5s_relu.rknn --target rk3588 --img_show

# If you want to run the original onnx model on the computer first, you can use the
following command
# Usage: python yolov5.py --model_path {onnx_model} --img_show
python yolov5.py --model_path ../model/yolov5s_relu.onnx --img_show
```

After the successful execution of the script, the output information will be as follows. In this context, the "class" field represents the predicted category, "score" is the confidence score, "(xmin, ymin)" are the coordinates of the top-left corner of the detection box, and "(xmax, ymax)" are the coordinates of the bottom-right corner of the detection box.

```
# class @ (xmin, ymin, xmax, ymax) score
person @ (209 244 286 506) 0.884
person @ (478 238 559 526) 0.868
person @ (110 238 230 534) 0.825
bus @ (94 129 553 468) 0.705
person @ (79 354 122 516) 0.339
```

The default input image is `model/bus.jpg` and the result image is as shown below:

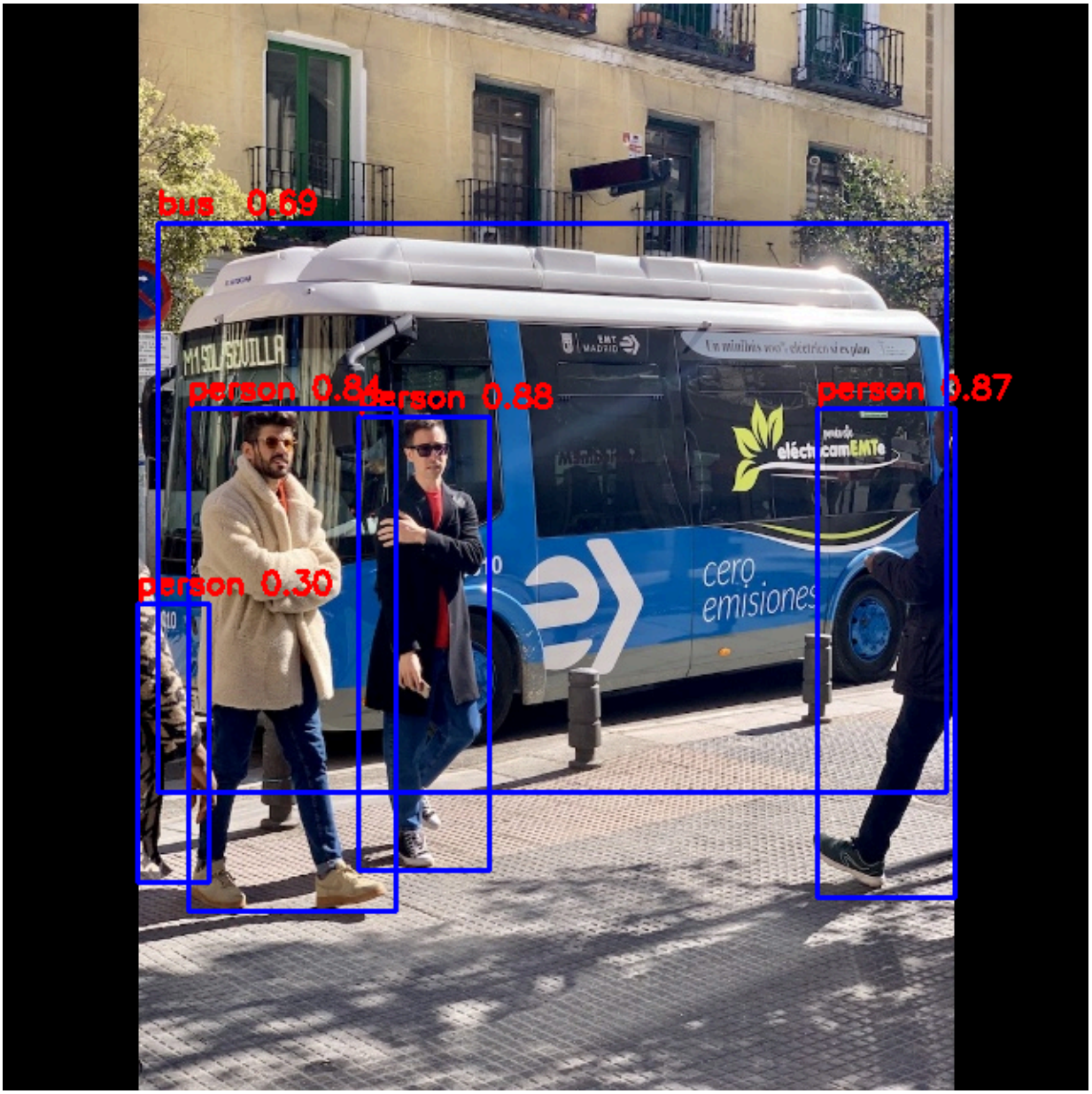

Figure 4-1 Visualization result of RKNN Python Demo

4.2.3 Run the RKNN C Demo

To run an RKNN C Demo, you need to first compile the C/C++ source code into an executable file, then push it to the board, and finally run it on the board. Before compiling, install the compilation tools first (see Section 3.4): configure ANDROID_NDK_PATH at the beginning of the build-android.sh script for Android boards, or GCC_COMPILER at the beginning of the build-linux.sh script for Linux boards. For the parameters of the build scripts, refer to the README.md in each example directory.

1. Compilation

```
# Board running on the Android system
# Note: configure ANDROID_NDK_PATH at the beginning of the build-android.sh script
cd Projects/rknn_model_zoo
./build-android.sh -t rk3588 -a arm64-v8a -d yolov5

# Board running on the Linux system
# Note: configure GCC_COMPILER at the beginning of the build-linux.sh script
cd Projects/rknn_model_zoo
./build-linux.sh -t rk3588 -a aarch64 -d yolov5
```

2. Push Files to the Board

After compilation, the executable file is located in the `rknn_yolov5_demo` folder under the `install` directory:

```
# Board running on the Android system
adb root
adb push install/rk3588_android_arm64-v8a/rknn_yolov5_demo /data/

# Board running on the Linux system
adb push install/rk3588_linux_aarch64/rknn_yolov5_demo /userdata/
```

3. Run the Demo on the Board

```
# Switch to the board
adb shell

# Switch to the rknn_yolov5_demo directory
# /data/rknn_yolov5_demo for Android, /userdata/rknn_yolov5_demo for Linux
cd /userdata/rknn_yolov5_demo

# Set the library dependency environment
export LD_LIBRARY_PATH=./lib

# Run the executable file
# Usage: ./rknn_yolov5_demo <model_path> <input_path>
./rknn_yolov5_demo model/yolov5s_relu.rknn model/bus.jpg
```

By default, the output image is saved at the path `rknn_yolov5_demo/out.png`. You can use the adb tool to pull it from the board to the local machine. In the local computer terminal, execute the following command:

```
# Pull to the local current directory
adb pull /userdata/rknn_yolov5_demo/out.png .
```

The output image is as follows:

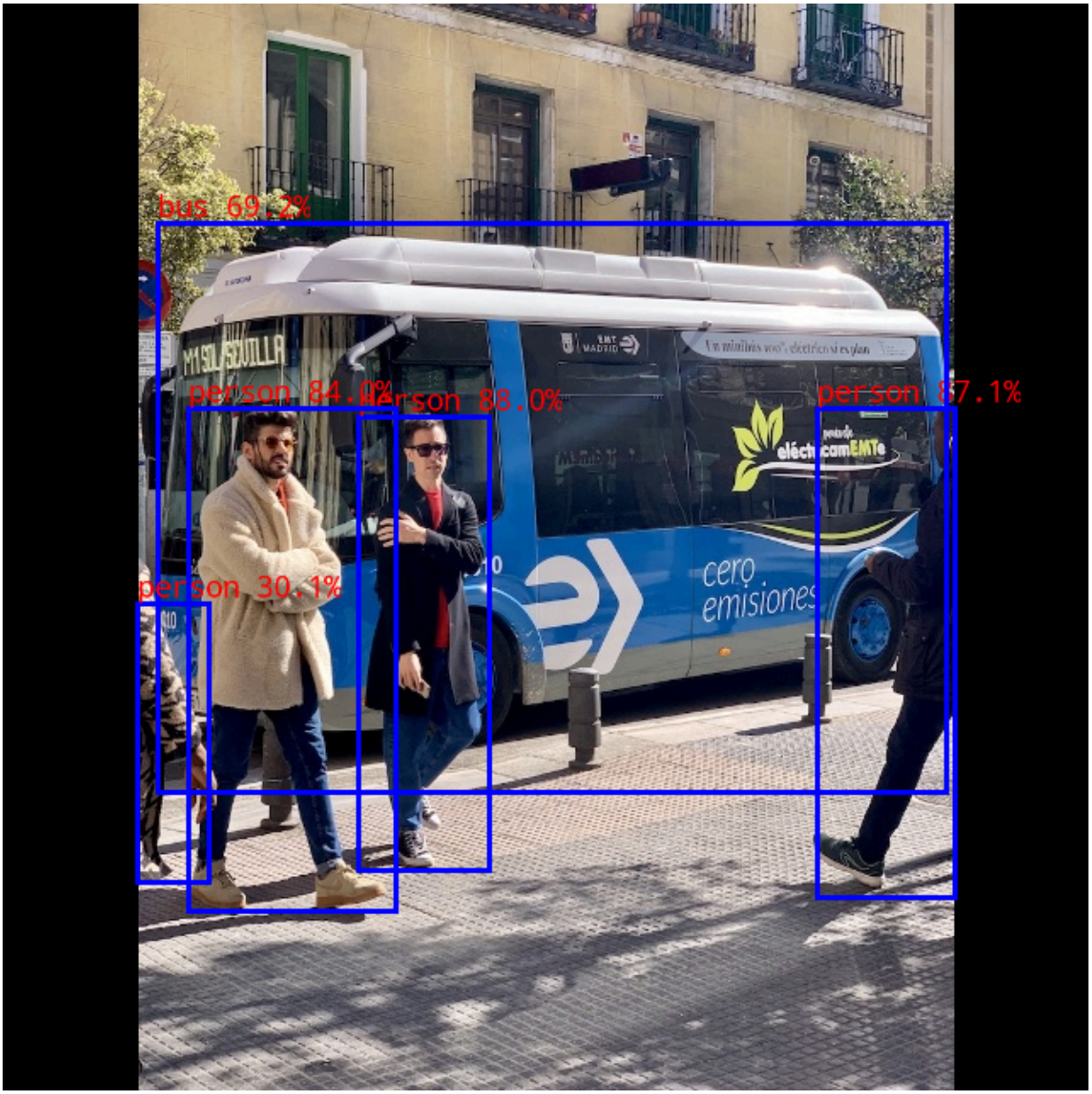

Figure 4-2 Visualization result of RKNN C Demo

For the first run, you only need to confirm that the Demo finishes normally and generates `out.png`. The actual detection results may vary with the model version, input image, and runtime environment.

4.2.4 Dataset Accuracy Evaluation (Optional)

The `rknn_model_zoo/datasets` directory stores datasets for accuracy evaluation. You need to download the evaluation dataset first and save it to this directory. For example, for the YOLOv5 model, the COCO dataset needs to be downloaded. Switch to the `rknn_model_zoo/datasets/COCO` directory and run the download_eval_dataset.py script. This script will download the val2017 dataset and store it in the current COCO directory. Refer to the following commands:

```
# Switch to the rknn_model_zoo/datasets/COCO directory
cd Projects/rknn_model_zoo/datasets/COCO

# Run the download_eval_dataset.py script to download the COCO dataset
python download_eval_dataset.py
```

When performing dataset accuracy evaluation, you need to specify the `--coco_map_test` parameter and the dataset path `--img_folder`. Refer to the following commands:

```
# Please install pycocotools first
pip install pycocotools

# Switch to the rknn_model_zoo/examples/yolov5/python directory
cd Projects/rknn_model_zoo/examples/yolov5/python

# Run the yolov5.py script
python yolov5.py \
    --model_path ../model/yolov5s_relu.rknn \
    --target rk3588 \
    --img_folder ../../../datasets/COCO/val2017 \
    --coco_map_test
```

> **Note:** For model-level accuracy analysis (comparing the inference results of the quantized model with those of the floating-point model layer by layer), use the `rknn.accuracy_analysis()` interface of RKNN-Toolkit2. For details, refer to the "Model Accuracy Analysis" section in the *RKNN2 SDK Development Guide* document.

5. Run RKNN Python Demo in Docker (Optional)

If you need to run the RKNN Python Demo in a Docker environment, please refer to the content in this chapter to prepare the development environment.

Please note that here we provide a Docker image containing the RKNN-Toolkit2 environment, allowing users to directly run the RKNN Python Demo without worrying about environment installation issues. However, this Docker image only includes a clean RKNN-Toolkit2 environment and is specifically designed for running the RKNN Python Demo.

5.1 Install Docker

If Docker is already installed, you can skip this step. If it is not installed, please follow the official documentation for installation.

Docker installation official documentation link: https://docs.docker.com/install/linux/docker-ce/ubuntu/

Note: It is necessary to add the user to the 'docker' group.

```
# Create the docker group
sudo groupadd docker

# Add the current user to the docker group
sudo usermod -aG docker $USER

# Update to activate the docker group
newgrp docker

# Verify that docker commands can be executed without sudo
docker run hello-world
```

The reference output for successful installation is as follows:

```
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
719385e32844: Pull complete
Digest: sha256:88ec0acaa3ec199d3b7eaf73588f4518c25f9d34f58ce9a0df68429c5af48e8d
Status: Downloaded newer image for hello-world:latest

Hello from Docker!
```

5.2 Prepare the RKNN-Toolkit2 Image

This section introduces two methods for creating the RKNN-Toolkit2 image environment. You can choose either method.

1. Create the RKNN-Toolkit2 image using a Dockerfile

In the RKNN-Toolkit2 project, the `docker/docker_file` folder provides a Dockerfile for building the RKNN-Toolkit2 development environment. Users can create an image using the `docker build` command, as shown below:

```
# Note: Replace 'xx' and 'x.x.x' with the actual version numbers
cd Projects/rknn-toolkit2/rknn-toolkit2/docker/docker_file/ubuntu_xx_xx_cpxx
docker build -f Dockerfile_ubuntu_xx_xx_for_cpxx -t rknn-toolkit2:x.x.x-cpxx .
```

2. Create the RKNN-Toolkit2 image by loading a pre-packaged Docker image file

Download the RKNN-Toolkit2 project files for the corresponding version from the following link. After extracting the files, the `docker/docker_image` folder provides a pre-packaged Docker image with all development environments.

Docker image file cloud drive download link: https://console.zbox.filez.com/l/I00fc3 (extraction code: rknn)

Execute the following command to load the image file for the corresponding Python version.

```
# Note: Replace 'x.x.x' with the actual version number of RKNN-Toolkit2, and 'cpxx'
with the Python version
docker load --input rknn-toolkit2-x.x.x-cpxx-docker.tar.gz
```

After the image is successfully created or loaded, you can view the Docker image information using the following command:

```
docker images
```

The corresponding RKNN-Toolkit2 image information is displayed as follows:

```
REPOSITORY    TAG        IMAGE ID     CREATED     SIZE
rknn-toolkit2 x.x.x-cpxx xxxxxxxxxxx 1 hours ago 5.89GB
```

5.3 Run the RKNN Python Demo in Docker

> **Note:** Please ensure that the board has the RKNPU2 environment installed. If not, please refer to Section 3.3 for installation instructions.

1. Map the files and run the container

Please refer to Section 1.5 to download the RKNN Model Zoo project locally. Then, map it into the container and run the container using the `docker run` command. After running, you will enter the container's bash environment. Refer to the following commands:

```
# Use the docker run command to create and run the RKNN-Toolkit2 container
# Map local files into the container by attaching the -v <host src folder>:<image
dst folder> parameter
docker run -t -i --privileged \
    -v /dev/bus/usb:/dev/bus/usb \
    -v /Projects/rknn_model_zoo:/rknn_model_zoo \
    rknn-toolkit2:x.x.x-cpxx \
    /bin/bash
```

2. Prepare the model and convert it in the container

After entering the container, follow the steps in Section 4.2.1 to download and convert the YOLOv5 model inside the container.

3. Run the RKNN Python Demo in the container

Follow the steps in Section 4.2.2 to run the RKNN Python Demo. Refer to the following command:

```
# Switch to the rknn_model_zoo/examples/yolov5/python directory
cd rknn_model_zoo/examples/yolov5/python

# Run the yolov5.py script to run the yolov5 model on the board
# Usage: python yolov5.py --model_path {rknn_model} --target {target_platform}
# Note: Here, we use rk3588 as an example platform. If using a different board,
modify the platform type in the command accordingly.
python yolov5.py --model_path ../model/yolov5s_relu.rknn --target rk3588
```

> **Note:** When using USB to connect the development board, please ensure that only one adb server service is running on the local computer and the Docker container at the same time. If you need to connect the development board in a Docker container, execute the command `adb kill-server` in the computer's terminal to terminate the adb server service on the local computer.

6. Common Startup Issues and Next Steps

This chapter lists the issues that may be encountered during the first run and the corresponding troubleshooting methods. For the complete FAQ, model conversion issues, accuracy issues, performance issues, and memory issues, please refer to Chapter 10 "FAQ" of the *RKNN2 SDK Development Guide* document.

6.1 adb devices Shows No Device

You can try the following methods to resolve this issue:

1. Check if the connection is correct, reconnect the data cable, try using a different USB port on the computer, or replace the data cable.
2. When using USB to connect the development board, please ensure that only one adb server service is running on the local computer and the Docker container at the same time. For example, if you need to connect the development board in a Docker container, execute the command `adb kill-server` in the computer's terminal to terminate the adb server service on the local computer.
3. If you encounter the following error, it indicates that adb is not installed. You need to execute the installation command `sudo apt install adb` to install adb.

```
command 'adb' not found, but can be installed with:
sudo apt install adb
```

6.2 Manually Start the rknn_server Service

The on-board debugging feature of RKNN-Toolkit2 requires the board to have the RKNPU2 environment installed and the rknn_server service running.

However, some development boards may not have the rknn_server service started by default after power on or reboot. This can result in errors such as `E init_runtime: The rknn_server on the connected device is abnormal` when running the RKNN Python Demo. In such cases, please refer to Section 3.3.2 for details on manually starting the rknn_server service.

6.3 Document Navigation

After completing the first run, please enter the corresponding main document according to your task, instead of looking for the complete API or advanced configuration in this quick start.

Content to explore further	Main document / entry
Complete RKNN-Toolkit2 installation, model conversion, and quantization configuration	*RKNN2 SDK Development Guide*

Content to explore further	Main document / entry
Toolkit methods, parameters, and return values (Python API)	*RKNN2 SDK Development Guide*
Runtime C API reference	*RKNN2 Runtime C API Reference*
Toolkit installation / NPU environment / model conversion / model evaluation FAQ	*RKNN2 SDK Development Guide*
Model accuracy analysis (accuracy_analysis)	*RKNN2 SDK Development Guide*
Docker-based RKNN-Toolkit2 installation	*RKNN2 SDK Development Guide*
Usage of each model example in RKNN Model Zoo	`rknn_model_zoo/examples/<model_name>/README.md`